



Build Steps

Part 1 of 5 · Set up your agent in the new builder

1 Create your agent

Open Copilot Studio (copilotstudio.microsoft.com), select **Create**, then **New agent**. In the redesigned builder everything lives under the **Build** tab, with a settings panel down the right. Set:

Agent name HR People Operations Copilot

Description Helps employees self-serve HR questions and request leave, and helps HR business partners get insights on the workforce.

Solution / Environment HR Solution · Default environment

Icon Use a recognisable colour and emoji, or upload a brand icon

2 Choose your model

Open the **Model** selector at the top of the right-hand panel. Copilot Studio now lets you pick the model your agent reasons with. This build uses **Claude Sonnet 4.6**; you can switch later without rebuilding.

Reasoning is built in

- The agent reasons over your instructions, knowledge, skills and tools, and decides what to do each turn
- Handles paraphrases, multi-step asks and follow-ups
- Blends a knowledge lookup with a tool call in one reply
- Calls the right skill when intent matches

Choosing a model

- Claude Sonnet 4.6: strong reasoning, a good default here
- Lighter models: lower latency and cost for simple, high-volume flows
- Switch the model and re-run the same evaluation set to compare
- Check data residency and model availability for your tenant

3 Write your instructions

The **Instructions** box is the heart of the Build tab. Define the agent's role, audiences and rules here. The agent reasons over these together with everything in the right-hand panel. Paste:

You are the HR People Operations Copilot. You help two audiences:

- 1) Employees asking about their own HR information.
- 2) HR business partners and managers analysing the workforce.

Rules:

- Use the uploaded employee data as your single source of truth.
- For action requests (book leave, raise a query), call the matching tool. Do not invent a confirmation if no tool returns success.
- For analytical questions, calculate from the rows and show your working.
- Treat employee data with care: never reveal another person's salary, rating, or PII to a non-HR user.
- If a question cannot be answered from data or tools, say so plainly.



Build Steps

Part 2 of 5 · Ground it in your data

4 Add Knowledge

Open **Knowledge** in the panel, select **Add knowledge**, then **File**, and upload **HR_People_Operations_Data.xlsx**. Knowledge gives the agent trusted context to guide its answers. This file contains:

- Employee details (name, department, role, location, manager)
- Employment data (hire date, status, salary band, backfill status)
- Leave data (balance, taken year-to-date)
- Performance and development (last review rating, training completion)

Layer more sources later: SharePoint sites, Dataverse, Microsoft Graph connectors, or public web.

5 Bring in Microsoft IQ

Open **Microsoft IQ** in the panel. Microsoft IQ brings in work context, business data and app signals from across your tenant, so the agent grounds answers in more than the file you uploaded.

For this agent, Microsoft IQ resolves the signed-in employee and their manager chain from the org graph, so 'who approves my leave' or 'route this to my manager' needs no manual lookup. Microsoft IQ honours each user's existing permissions, so the agent only surfaces what the signed-in user is already allowed to see.

Build Steps

Part 3 of 5 · Give it skills and tools

6 Add a Skill

Open **Skills** in the panel, then **Add a skill**. A skill defines a reusable behaviour through structured instructions. It is the new home for the conversational flows you used to build as topics: the agent calls the skill when a user's intent matches.

Skill name: Request Annual Leave

Description: Guides an employee through booking annual leave: collects dates and leave type, then logs the request via Power Automate.

When to use it (intent hints the agent reasons over):

- I want to book holiday
- Request annual leave
- Book time off next week
- How do I take leave

Structured steps:

- Ask for the leave type (Annual / Sick / Parental / Unpaid)
- Ask for start and end dates
- Look up the employee's leave balance from knowledge and confirm
- Call the 'Submit Leave Request' tool with the collected inputs
- Confirm the submission ID returned by the tool



7 Add a Tool

Open **Tools**, then **Add a tool**. **Skills decide what to do; tools take the action**. A tool is a typed, callable action with named inputs and outputs that the model fills from the conversation. Choose one of: a **prebuilt connector** (Outlook, Teams, SharePoint, Dataverse, ServiceNow and more), a **Power Automate cloud flow**, a **custom HTTP request**, or a **Model Context Protocol (MCP) server**. This build uses a Power Automate flow because it does two things in one call: log the request and post an approval card to the manager.

Tool name: Submit Leave Request

Type: Power Automate flow with an HTTP trigger that writes to a SharePoint list 'Leave Requests'

Description (the model reads this to decide when to call): Logs a new leave request and notifies the employee's line manager. Use when an employee asks to book or request time off.

Inputs the agent collects or infers:

- employeeId — text, from the authenticated user
- leaveType — text, one of Annual / Sick / Parental / Unpaid
- startDate — date, first day of leave
- endDate — date, last day of leave
- note — text, optional reason or context

Outputs returned to the agent:

- submissionId — text, e.g. LR-20260608-0142
- status — text, e.g. Submitted
- approvalCardPosted — yes/no

7.1 · Build the cloud flow that does the work

From inside Copilot Studio choose **Tools > Add a tool > New > Power Automate**. This opens the flow designer pre-wired with the Copilot Studio trigger and, crucially, keeps the flow in the **same environment** as your agent (HR Solution · Default). An environment mismatch is the most common reason a finished tool never appears in the picker, so confirm the environment switcher (top-right) before you build.

Trigger — *When an agent calls the flow*. Add five input parameters, named exactly: employeeId, leaveType, startDate, endDate, note.

– **Action 1 – generate the submission ID.** Add *Initialize variable* `submissionId = concat('LR-', formatDateTime(utcNow(), 'yyyyMMdd'), '-', rand(1000, 9999))`. (Or let Dataverse / SharePoint return the ID instead.)

– **Action 2 – write the request.** SharePoint *Create item* (or Dataverse *Add a new row*) into the **Leave Requests** list. Map: Title = submissionId, Employee = employeeId, LeaveType = leaveType, Start = startDate, End = endDate, Note = note, Status = 'Pending Approval'.

– **Action 3 – notify the manager.** Microsoft Teams *Post card in a chat or channel* to the employee's line manager, using an adaptive card that shows the submission ID, employee, leave type and dates with Approve / Decline actions.

– **Final action – return to the agent.** Add *Respond to the agent* (Return value(s) to Copilot Studio) with outputs `submissionId = variables('submissionId')`, `status = 'Submitted'`, `approvalCardPosted = true`.

– **Save** the flow as HR – Submit Leave Request.



7.2 · Register and map the tool in your agent

- Back in Copilot Studio the flow is attached automatically; if you built it separately, **Tools > Add a tool > Flows** and select it.
- Set the display name to **Submit Leave Request** and write the **Description** for the model — it is what the agent reads to decide when to call the tool.
- For each input set **Fill using**: employeeId → *dynamically* from System . User . Id; leaveType, startDate, endDate and note → **Fill with AI**, and add an input description so the model extracts the right value (for leaveType, list Annual / Sick / Parental / Unpaid and what each means).
- Confirm submissionId is returned to the agent so the skill can read it back — satisfying your instruction rule “do not invent a confirmation if no tool returns success”.
- **Connection & auth**: set the SharePoint and Teams connections the flow uses. The connectors chosen here are exactly what your DLP policy must permit (see Step 10).

7.3 · Worked example — what flows in and out

When a user says “book me 3 days off from next Monday”, the model fills the inputs and calls the tool with this payload:

```
{
  "employeeId": "E-10293",
  "leaveType": "Annual",
  "startDate": "2026-06-15",
  "endDate": "2026-06-17",
  "note": "Long weekend"
}
```

The flow writes the row, posts the approval card, and returns:

```
{
  "submissionId": "LR-20260608-0142",
  "status": "Submitted",
  "approvalCardPosted": true
}
```

The agent then replies in natural language, quoting the real reference: “I’ve submitted your annual leave request LR-20260608-0142 for 15–17 Jun and sent an approval card to your manager.”

7.4 · Test the tool before you trust it

- **In isolation**: open the flow in Power Automate, choose *Test > Manually*, supply the five inputs, run it, and confirm a row appears in the Leave Requests list, an approval card posts to the manager, and the run returns all three outputs.
- **End to end**: in **Preview** (Step 11) type “book 3 days of leave for me starting next Monday” and check the activity view shows Submit Leave Request firing with the inputs the model inferred and the submission ID it read back.



Build Steps

Part 4 of 5 · Extend, remember and govern

8 Add Connected agents

Open **Connected agents**. A connected agent lets this agent collaborate with others, delegating specialist work and sharing context across them (agent-to-agent). Keep each agent focused on what it does best.

Connect a **Payroll Agent** for pay and benefits questions. Keep it separate so the people-ops agent never touches pay data directly, and hands those questions off only to a user entitled to ask them.

9 Turn on Memory

Toggle **Memory** on. Memory lets the agent remember interactions, workflows and context across sessions, so returning users do not have to repeat themselves. For this agent it remembers:

- An employee's recent leave requests, so follow-ups stay in context
- A manager's team, so 'show my team's training gaps' resolves without re-stating it

Memory is scoped to each user and follows your retention and DLP policy. Review what the agent retains before you publish.

10 Govern your agent

Open the agent settings and lock these down before real users arrive:

Authentication: use *Authenticate with Microsoft (Entra)* so the agent runs as the signed-in user and inherits their permissions.

Content moderation: set the moderation level (start at Medium) and review it for your audience.

Data loss prevention: confirm your DLP policy covers every connector your tools use — for Submit Leave Request that means the SharePoint and Teams connectors must sit in the same DLP group, or the tool is blocked at runtime.

Channels and access: review which channels are enabled and who can use the agent.

Build Steps

Part 5 of 5 · Preview, evaluate and publish

11 Preview your agent

Switch to the **Preview** tab to chat with your agent as you build. Open the **activity view** on any response to inspect:

- Which skill, tool or knowledge source fired, and in what order
- The inputs the agent collected and the exact payload each tool returned
- Citations linking each answer back to a row of your data
- Latency per step, useful when tuning multi-tool flows



12 Evaluate

Open the **Evaluate** tab, then **New evaluation**. Build a test set so you can measure quality before and after every change. Score each response on **groundedness** (did it stay inside the knowledge), **relevance** (did it address the question), and **accuracy** (did it match the expected outcome).

Test set: HR Copilot · Baseline Evaluation v1

Input: How many days of annual leave do I have left?

Expect: Returns the employee's own balance only, never another person's.

Input: Which employees have a Developing review rating?

Expect: Lists employees only when the user is identified as HR or a manager.

Input: Book 3 days of leave for me starting next Monday

Expect: Triggers the Request Annual Leave skill and calls the Submit Leave Request tool.

13 Publish and Monitor

Click **Publish**, then enable the channels your users live in: Microsoft Teams, Microsoft 365 Copilot, a custom website embed, or other channels. Teams and M365 Copilot need tenant admin approval, so plan for that lead time. After publishing, open the **Monitor** tab to watch sessions, resolution and escalation outcomes, and quality trends over time, and to see where users drop off.

Try These Live

Sample queries to test your agent during the build-along.

Self-service (Employee)

How many days of annual leave do I have left? · Book 3 days of annual leave for me starting next Monday.

Workforce Insights (HR / Manager)

Show headcount by department, highest to lowest. · Which employees are below 80% training completion?

Tool Validation

Submit a leave request and check the Teams approval card arrived.

Try a paraphrase the skill was not trained on, e.g. 'I'm off Friday'.

How to Think About This Agent

This agent replaces	With
• HR email queues for routine leave questions • Manual leave-form processing • Repeated workforce reports for managers	✓ Self-service for employees, 24/7 ✓ Actions taken safely via governed tools ✓ Faster, evidence-led HR conversations

Trusted, governed, enterprise AI. Built once, used everywhere.



Tip for real-world use

Bind the agent to the signed-in identity via Entra so an employee only ever sees their own record. Keep an evaluation set of your top 20 self-service questions so a model or prompt change never quietly leaks another person's data.

You've built a Copilot Studio agent ✓

You now have an AI agent that can:

- Answer self-service HR questions
- Book leave via a governed tool
- Give managers workforce insight
- Cut routine load on the HR team